COMPSCI 572.01 (NLP)

Efficient Training & Inference

The New York Times

NEWSLETTER

Climate Forward

A.I. Could Soon Need as Much Electricity as an Entire Country

Behind the scenes, the technology relies on thousands of specialized computer chips.

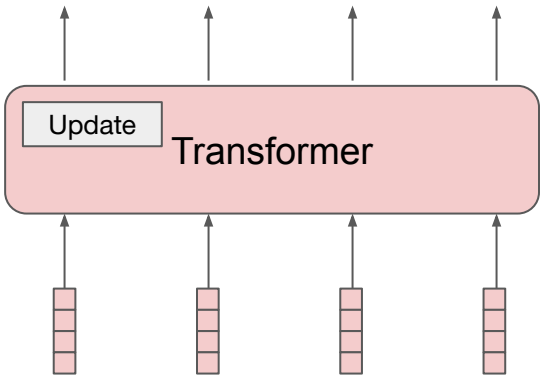
NYTimes, October 2023

Overview

- Training efficiency:
  - Parameter-efficient finetuning
    - Prefix tuning
    - LoRA
    - Adapters
- Inference efficiency:
  - Speculative decoding
  - Knowledge distillation

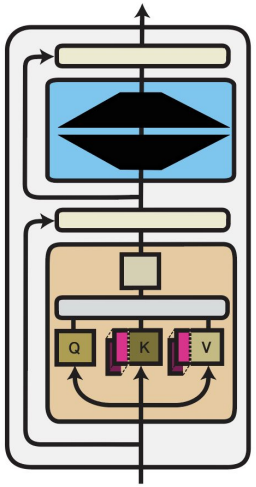
LLM training pipeline

Pretraining → **Supervised Finetuning** → **RLHF / RLVR**

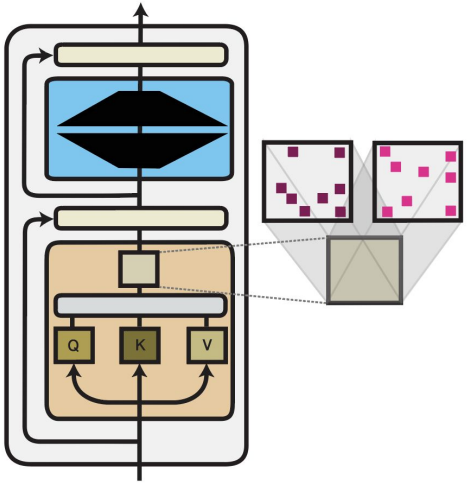


- **Full finetuning:** all parameters are updated in each stage of training
- **Parameter efficient finetuning (PEFT):** update a small subset of model parameters

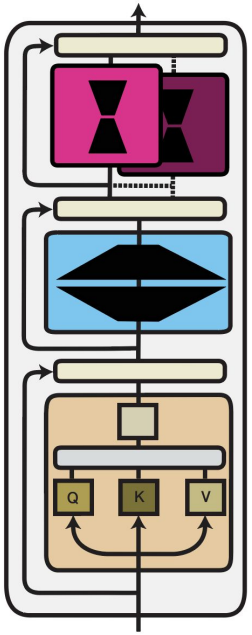
Three types of PEFT techniques



At the level of **inputs**



At the level of **parameters**

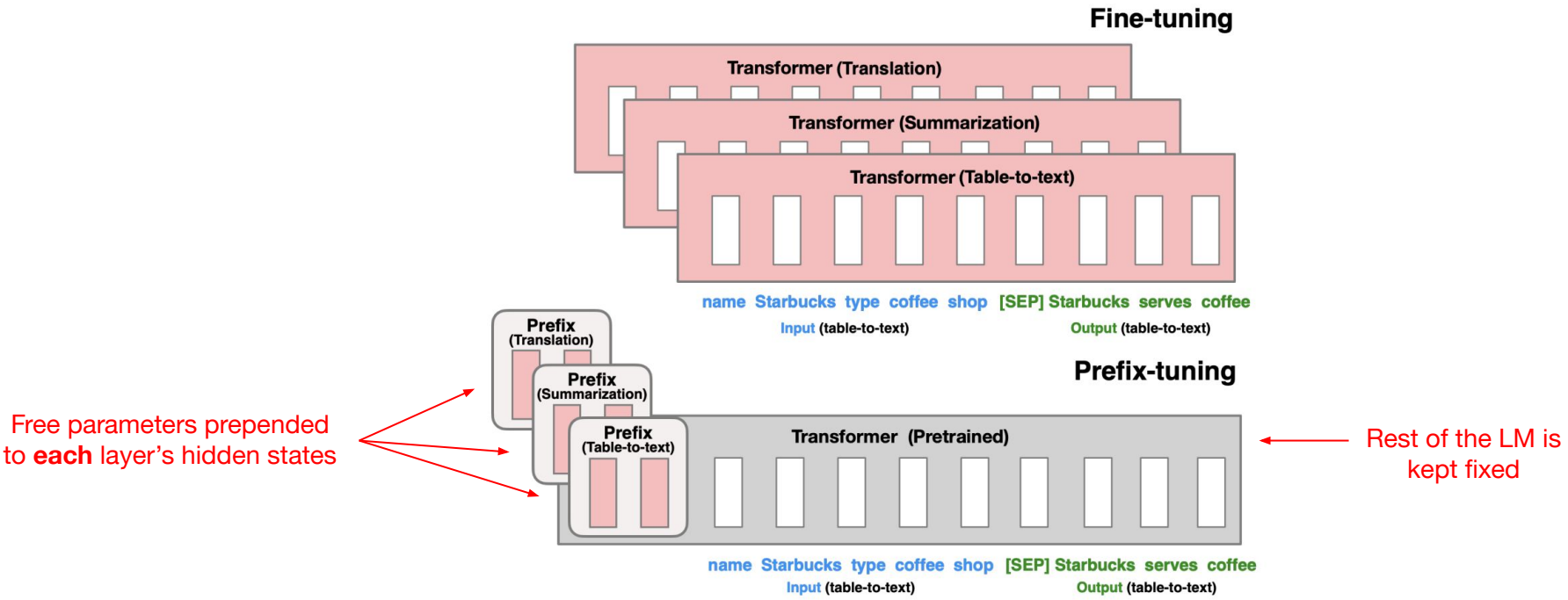


At the level of **functions**

Slide credit: Sebastian Ruder and others

At the input level: Prefix tuning

Idea: Optimize task-specific prefix vectors which induce the correct labels



Li et al; 2021

Prefix tuning

When the training data size is limited, this can perform better than full finetuning

Summarization

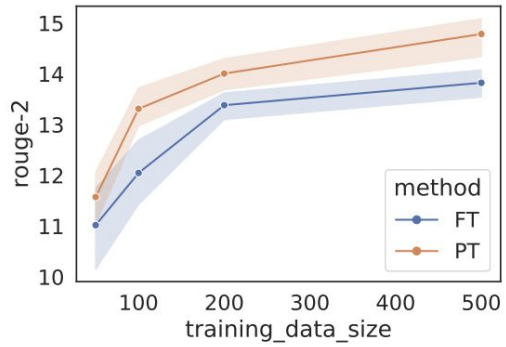
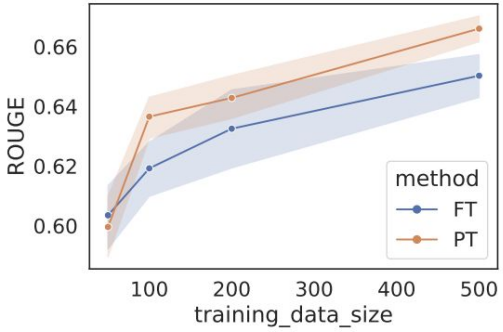
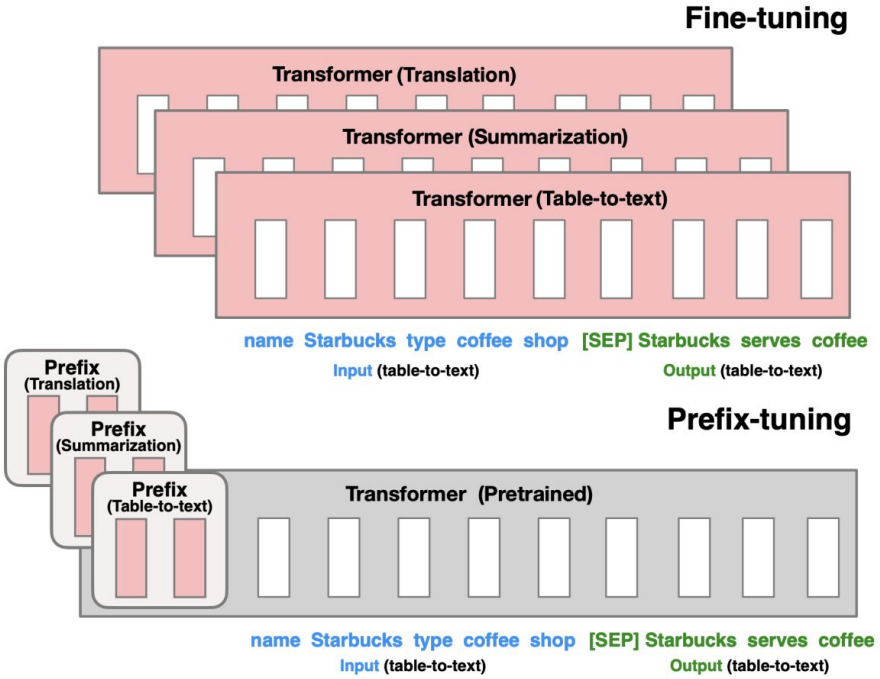


Table-to-text





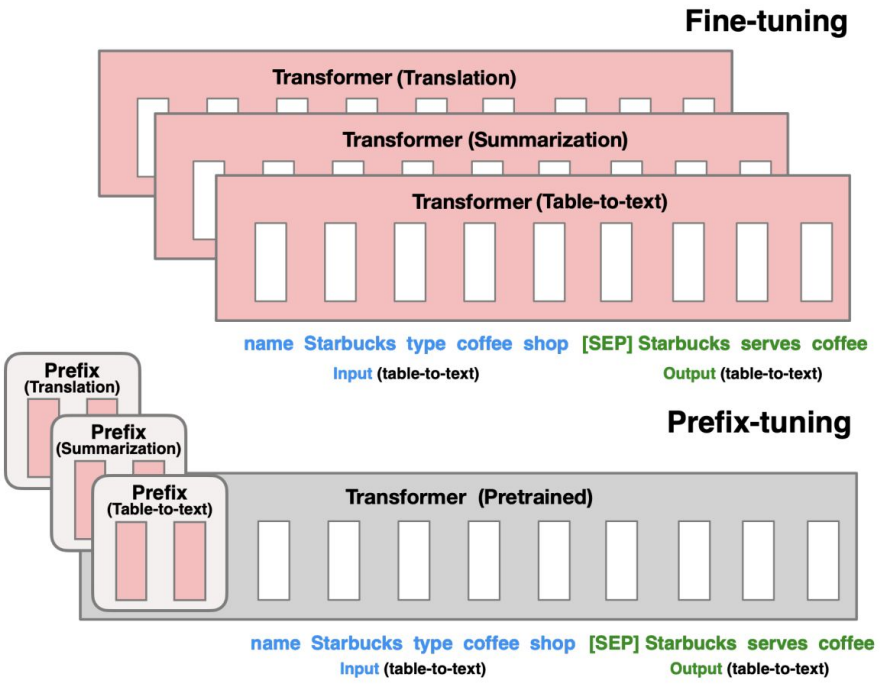
Li et al; 2021

Prefix tuning

Do we need to add the prefix vectors to every layer?

	E2E				
	BLEU	NIST	MET	ROUGE	CIDEr
PREFIX	69.7	8.81	46.1	71.4	2.49
Embedding-only: EMB-{PrefixLength}					
EMB-1	48.1	3.33	32.1	60.2	1.10
EMB-10	62.2	6.70	38.6	66.4	1.75
EMB-20	61.9	7.11	39.3	65.6	1.85

Only adds prefix vectors to the bottom (embedding) layer

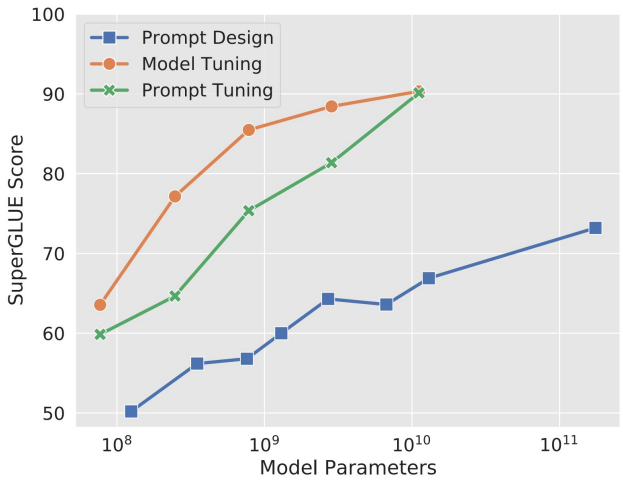


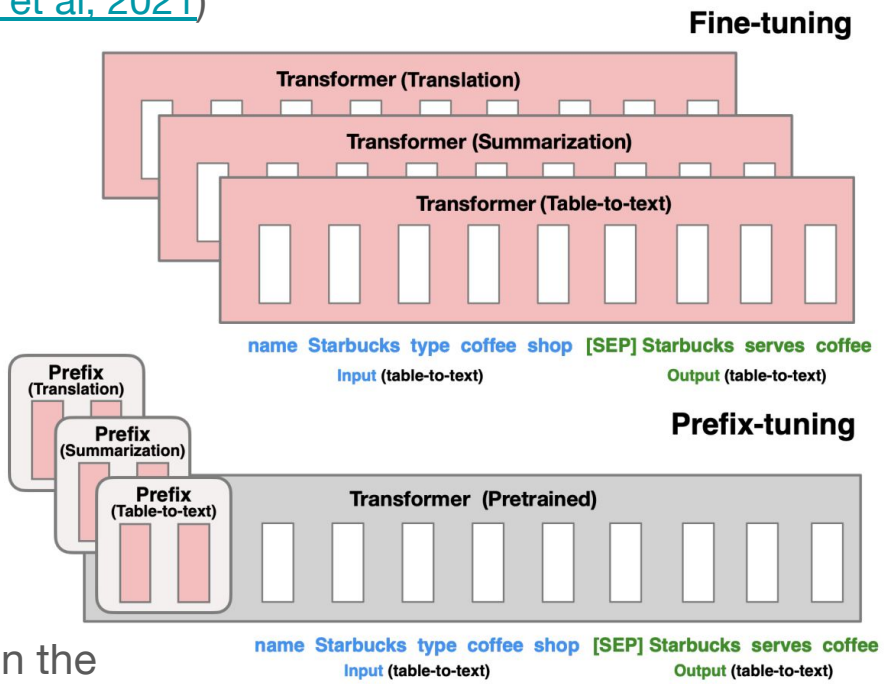
Li et al; 2021

Prefix tuning

In practice, much simpler to only add prefixes to the first layer

- This becomes competitive with scale (Lester et al; 2021)

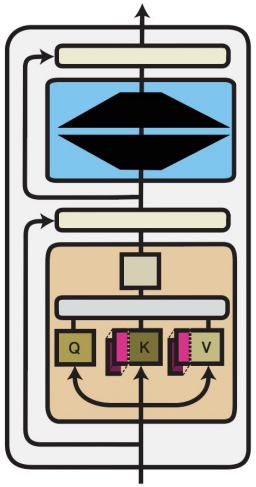




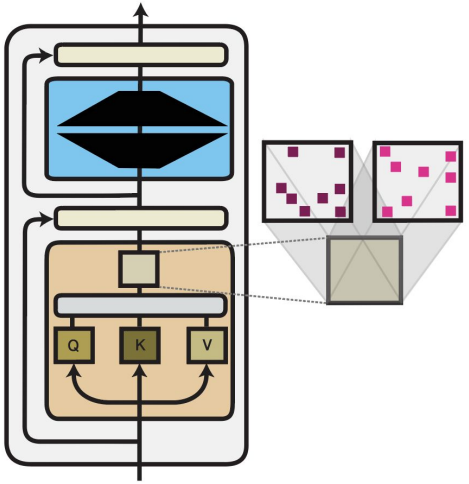
Another advantage: can process multiple tasks in the same batch

Li et al; 2021

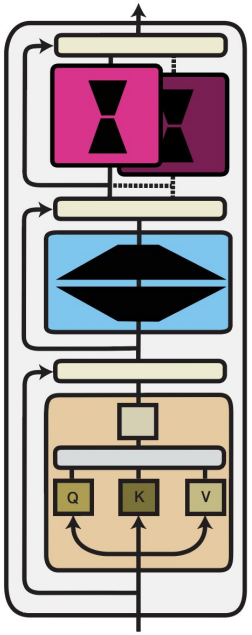
Three types of PEFT techniques



At the level of **inputs**



At the level of **parameters**



At the level of **functions**

Slide credit: Sebastian Ruder and others

Low Rank Adaptation (LoRA)

- During finetuning we update the pretrained parameters of the LM θ_0 to $\theta_0 + \Delta\theta$, by optimizing the following objective:

$$\max_\theta \sum_{(x,y)} \sum_{t=1}^{|y|} \log p_\theta(y_t | x, y_{<t})$$

- For different tasks, we learn different $\Delta\theta$, *which are all the same size as* θ_0

Hu et al; 2022

Low Rank Adaptation (LoRA)

- During finetuning we update the pretrained parameters of the LM θ_0 to $\theta_0 + \Delta\theta$, by optimizing the following objective:

$$\max_\theta \sum_{(x,y)} \sum_{t=1}^{|y|} \log p_\theta(y_t | x, y_{<t})$$

- For different tasks, we learn different $\Delta\theta$, *which are all the same size as* θ_0

- **Idea:** Encode $\Delta\theta = f(\phi)$ as a function of a *smaller set* of parameters ϕ and learn them:

$$\max_\phi \sum_{(x,y)} \sum_{t=1}^{|y|} \log p_{\theta_0 + f(\phi)}(y_t | x, y_{<t})$$

$|\phi| \ll |\theta|$

Initial LM parameters

Parameter update

Hu et al; 2022

Low Rank Adaptation (LoRA)

- For each pretrained weight matrix

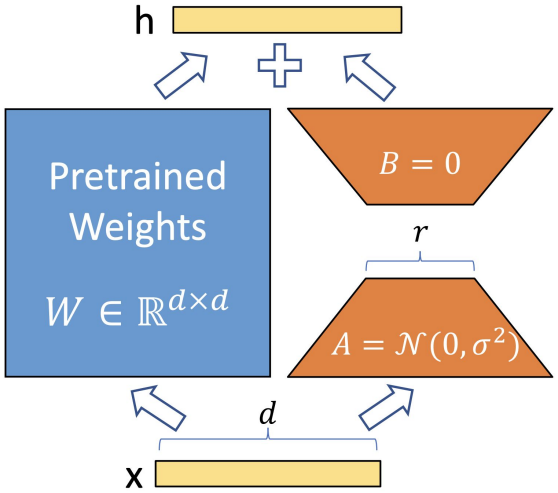
$$W_0 \in \mathbb{R}^{d \times k}$$

- Learn an update $W_0 + \Delta W = W_0 + BA$ where:

$$B \in \mathbb{R}^{d \times r}, A \in \mathbb{R}^{r \times k}, r \ll \min(d, k)$$

(low-rank decomposition)

- Finetune only A, B (total $r(d+k)$ params)

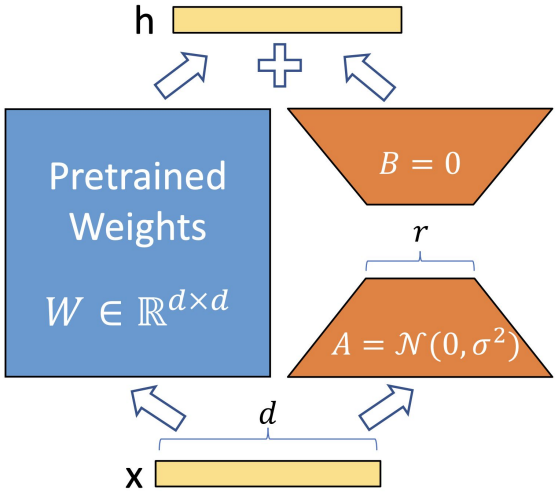


Hu et al; 2022

Low Rank Adaptation (LoRA)

- Increasing r smoothly approaches full finetuning
- **No additional inference latency:** forward passes are involve exactly the same amount of compute
- **Can still switch between tasks easily:** just subtract $B_1 A_1$ and add $B_2 A_2$

Tinker is a library for doing basically exactly this



Hu et al; 2022

Low Rank Adaptation (LoRA)

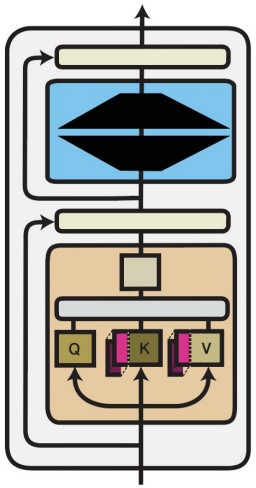
- LoRA works surprisingly well:

	Model&Method	# Trainable Parameters	WikiSQL Acc. (%)	MNLI-m Acc. (%)	SAMSum R1/R2/RL
Full finetuning →	GPT-3 (FT)	175,255.8M	**73.8**	89.5	52.0/28.0/44.5
	GPT-3 (BitFit)	14.2M	71.3	91.0	51.3/27.4/43.5
	GPT-3 (PreEmbed)	3.2M	63.1	88.6	48.3/24.2/40.5
	GPT-3 (PreLayer)	20.2M	70.1	89.5	50.8/27.3/43.5
	GPT-3 ($\text{Adapter}^{\text{H}}$)	7.1M	71.9	89.8	53.0/28.9/44.8
	GPT-3 ($\text{Adapter}^{\text{H}}$)	40.1M	73.2	**91.5**	53.2/29.0/45.1
LoRA →	GPT-3 (LoRA)	4.7M	73.4	**91.7**	**53.8/29.8/45.9**
	GPT-3 (LoRA)	37.7M	**74.0**	**91.6**	53.4/29.2/45.1

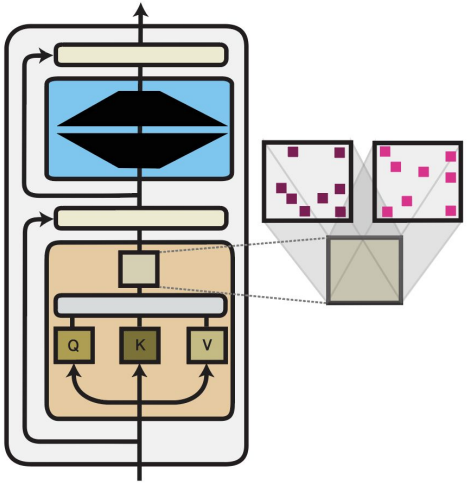
- LoRA has been quite successful on recent LLMs too

Hu et al; 2022

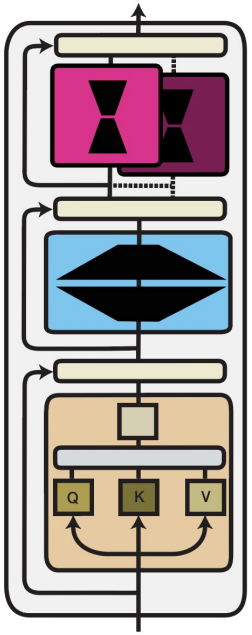
Three types of PEFT techniques



At the level of **inputs**
(Prefix Tuning)



At the level of **parameters**
(LoRA)



At the level of **functions**

Slide credit: Sebastian Ruder and others

Adapters

- **Idea**: augment a model with new task-specific layers

$$f(x) = \left(f_\theta \odot f_{\phi_i}\right)(x)$$

Function composition

Adapter for task i

Houlsby et al; 2022

Adapters

- **Idea**: augment a model with new task-specific layers

$$f(x) = \left(f_\theta \odot f_{\phi_i}\right)(x)$$

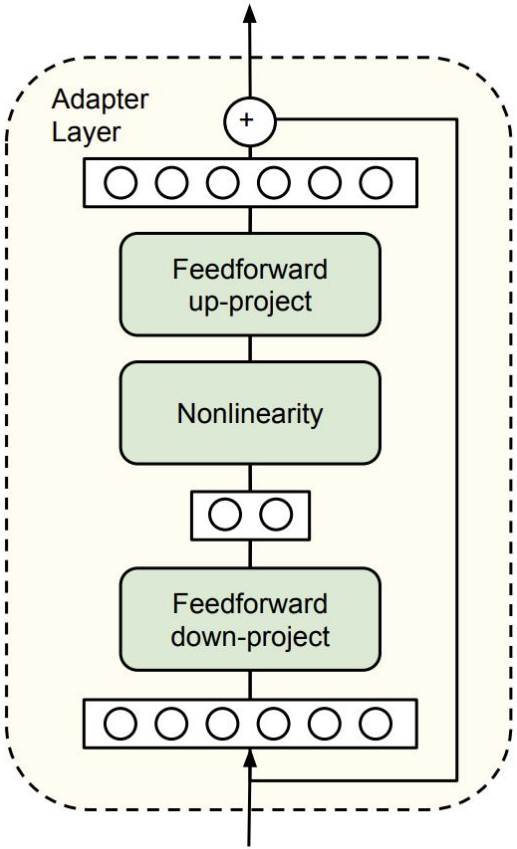
Function composition

Adapter for task i

- For transformers, adapters generally consist of:
  - A feed-forward down-projection $W_D \in \mathbb{R}^{d \times k}$
  - A feed-forward up-projection $W_U \in \mathbb{R}^{k \times d}$

$$f_{\phi_i}(x) = W_U \sigma(W_D x)$$

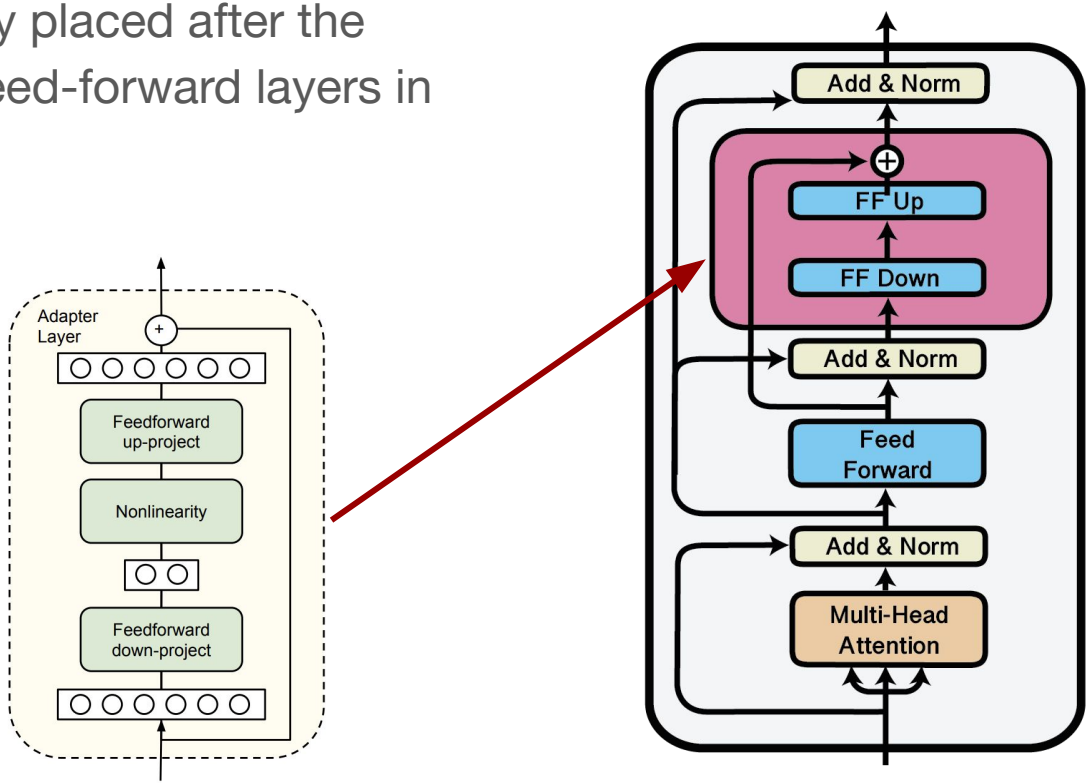
Non-linearity



Houlsby et al; 2022

Adapters

- Adapter layers are usually placed after the multi-head attention or feed-forward layers in each Transformer layer

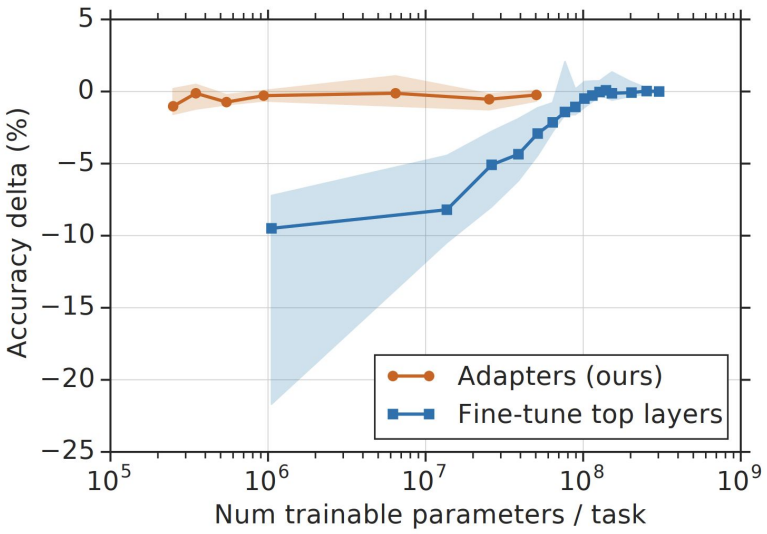


Houlsby et al; 2022

Adapters

- Adapters are more parameter efficient than freezing Transformer layers
  - Results on BERT

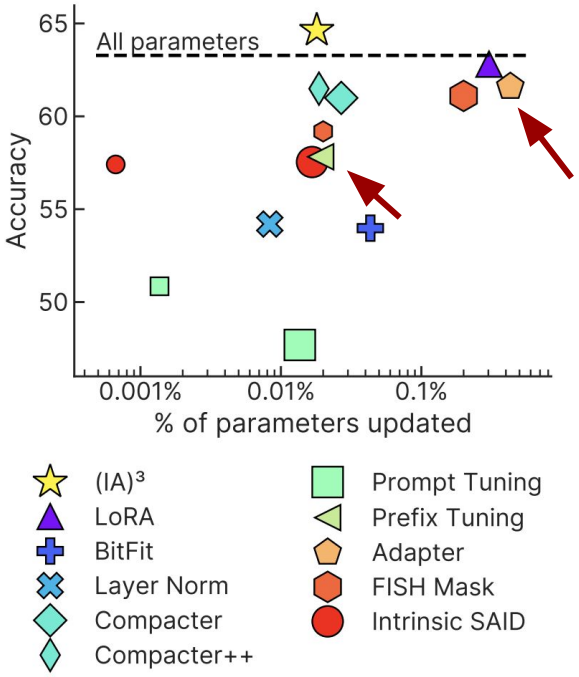
Relative accuracy compared to full finetuning



Measured across 9 GLUE benchmark tasks

Houlsby et al; 2022

Performance comparisons



These days, LoRA is the most common method used.

He et al, 2022
Liu et al; 2022

Overview

- Training efficiency:
  - Parameter-efficient finetuning
    - Prefix tuning
    - LoRA
    - Adapters
- Inference efficiency:
  - Speculative decoding
  - Knowledge distillation

LLM Inference

Autoregressive models generate text by sequentially sampling the next token:

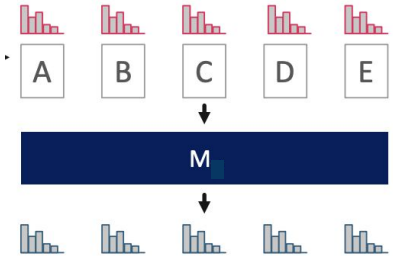
$$\underbrace{t_i}_{\text{Next token}} \sim \underbrace{P_M}_{\text{Language model}}(t|\underbrace{t_1, t_2, \ldots, t_{i-1}}_{\text{Response generated so far}}, \underbrace{q}_{\text{Input Prompt}})$$

LLM Inference

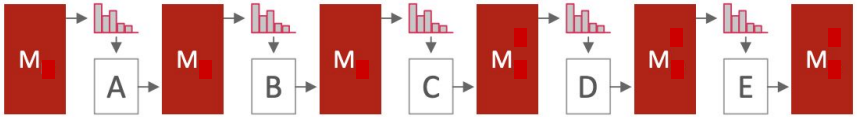
Autoregressive models generate text by sequentially sampling the next token:

$$\underbrace{t_i}_{\text{Next token}} \sim \underbrace{P_M}_{\text{Language model}}(t|\underbrace{t_1, t_2, \ldots, t_{i-1}}_{\text{Response generated so far}}, \underbrace{q}_{\text{Input Prompt}})$$

Processing the input prompt is faster than generating the response



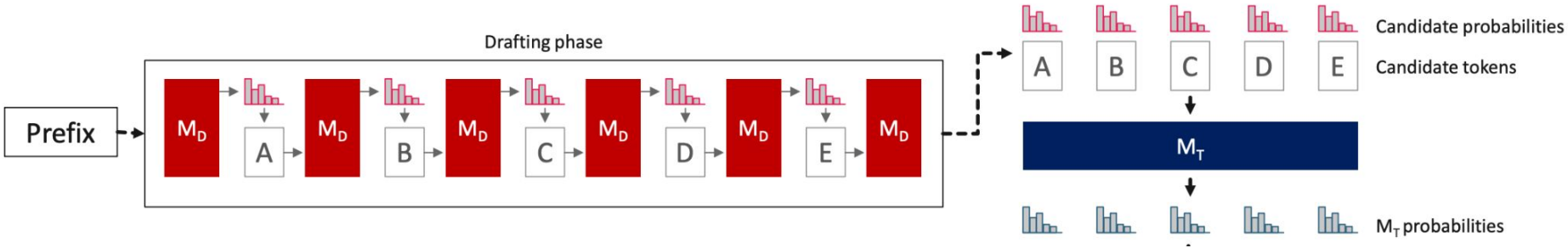
Prefill (Parallel)



Decode (Sequential)

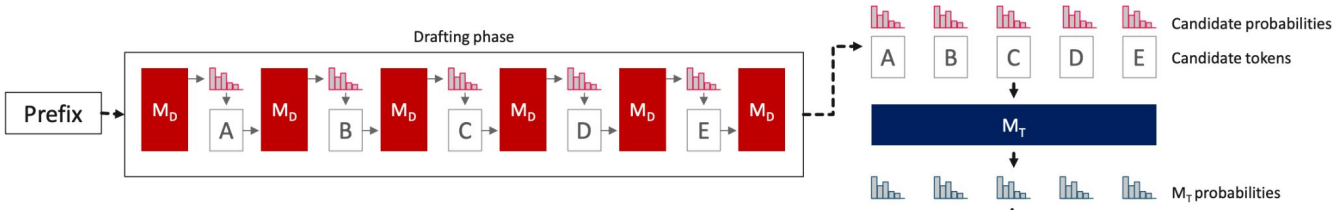
Speculative decoding (Leviathan et al; 2022)

Key Idea: Smaller "draft" model M_D generates sequentially and larger "target" model M_T verifies in parallel



Fast Inference from Transformers via Speculative Decoding, Leviathan et al, 2022; https://arxiv.org/abs/2211.17192

Speculative decoding (Leviathan et al; 2022)

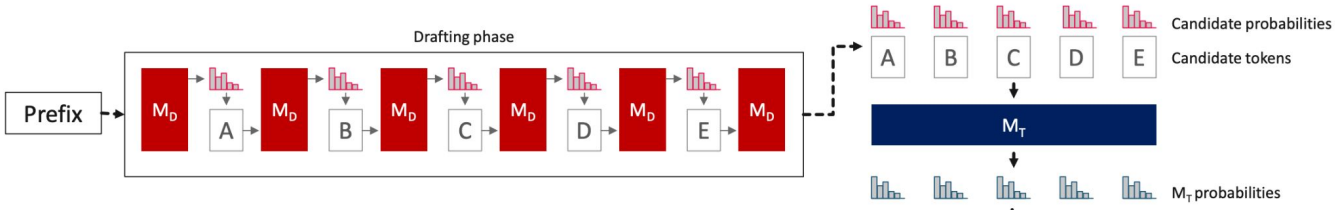


For candidate A from A B C D E :

Rejection sampling for token acceptance

Fast Inference from Transformers via Speculative Decoding, Leviathan et al, 2023; https://arxiv.org/abs/2211.17192

Speculative decoding (Leviathan et al; 2022)



For candidate A from A B C D E :

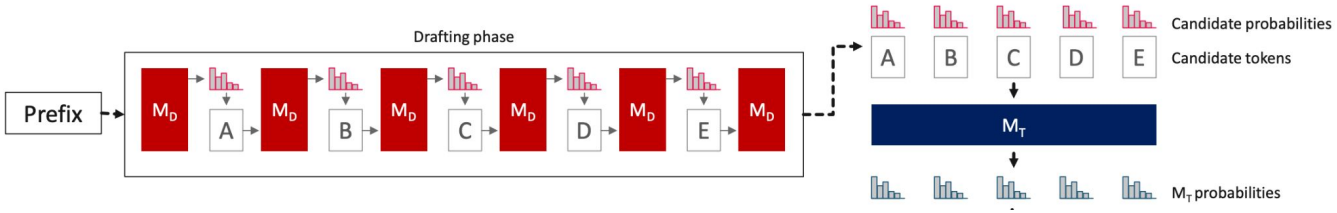
If (A) < (A) → accept A

Else, accept A with probability $\frac{(A)}{(A)}$

Rejection sampling for token acceptance

Fast Inference from Transformers via Speculative Decoding, Leviathan et al, 2023; https://arxiv.org/abs/2211.17192

Speculative decoding (Leviathan et al; 2022)



For candidate A from A B C D E :

If (A) < (A) → accept A

Else, accept A with probability $\frac{(A)}{(A)}$

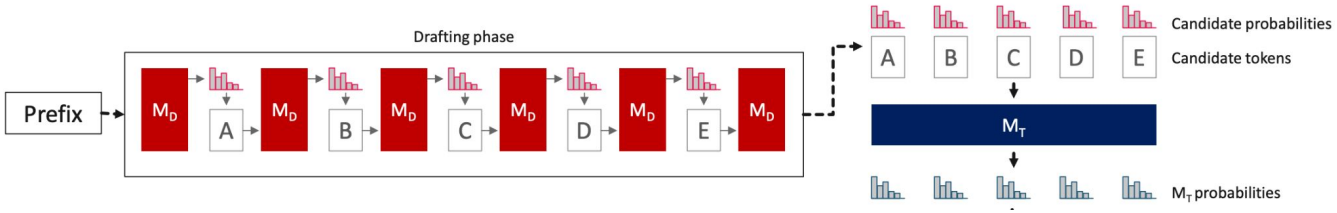
If A reject, resample from − =

(repeat until first rejection)

Rejection sampling for token acceptance

Fast Inference from Transformers via Speculative Decoding, Leviathan et al, 2023; https://arxiv.org/abs/2211.17192

Speculative decoding (Leviathan et al; 2022)



For candidate A from A B C D E :

If (A) < (A) → accept A

Else, accept A with probability $\frac{(A)}{(A)}$

If A reject, resample from − =

(repeat until first rejection)

Rejection sampling for token acceptance

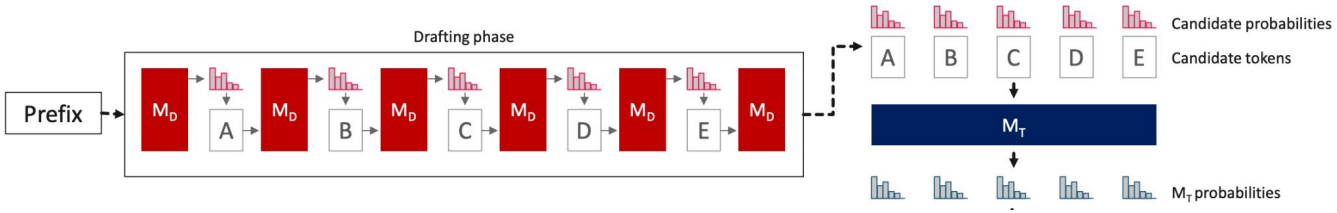
A B C → F ~ → A B F

Accept Accept Reject Resample Final accepted sequence

→ Send back to draft model for generating next set of candidates

Fast Inference from Transformers via Speculative Decoding, Leviathan et al, 2023; https://arxiv.org/abs/2211.17192

Speculative decoding (Leviathan et al; 2022)



For candidate A from A B C D E :

If (A) < (A) → accept A

Else, accept A with probability $\frac{(A)}{(A)}$

If A reject, resample from − =

(repeat until first rejection)

Rejection sampling for token acceptance

⇒ Guarantees distributional equivalence to M_T

Send back to draft model for generating next set of candidates

Fast Inference from Transformers via Speculative Decoding, Leviathan et al, 2023; https://arxiv.org/abs/2211.17192

Why does this work?

A.1. Correctness of Speculative Sampling

We will now show that for any distributions $p(x)$ and $q(x)$, the tokens sampled via *speculative sampling* from $p(x)$ and $q(x)$ are distributed identically to those sampled from $p(x)$ alone. Let β be the acceptance probability (Definition 3.1).

Note that as $p'(x) = norm(max(0, p(x) - q(x))) = \frac{p(x) - min(q(x), p(x))}{\sum_{x'}(p(x') - min(q(x'), p(x')))} = \frac{p(x) - min(q(x), p(x))}{1-\beta}$, the normalizing constant for the adjusted distribution $p'(x)$ is $1 - \beta$, where the last equation follows immediately from Lemma 3.3 and Theorem 3.5.

Now:

$$P(x = x') = P(guess\ accepted, x = x') + P(guess\ rejected, x = x')$$

Where:

$$P(guess\ accepted, x = x') = q(x') \min(1, \frac{p(x')}{q(x')}) = \min(q(x'), p(x'))$$

And:

$$P(guess\ rejected, x = x') = (1 - \beta)p'(x') = p(x') - \min(q(x'), p(x'))$$

Overall:

$$P(x = x') = \min(p(x'), q(x')) + p(x') - \min(p(x'), q(x')) = p(x').$$

As desired. □

Proof. $\beta = E_{x \sim q(x)} \begin{cases} 1 & q(x) \leq p(x) \\ \frac{p(x)}{q(x)} & q(x) > p(x) \end{cases}$

$E_{x \sim q(x)} \min(1, \frac{p(x)}{q(x)}) = \sum_x \min(p(x), q(x))$

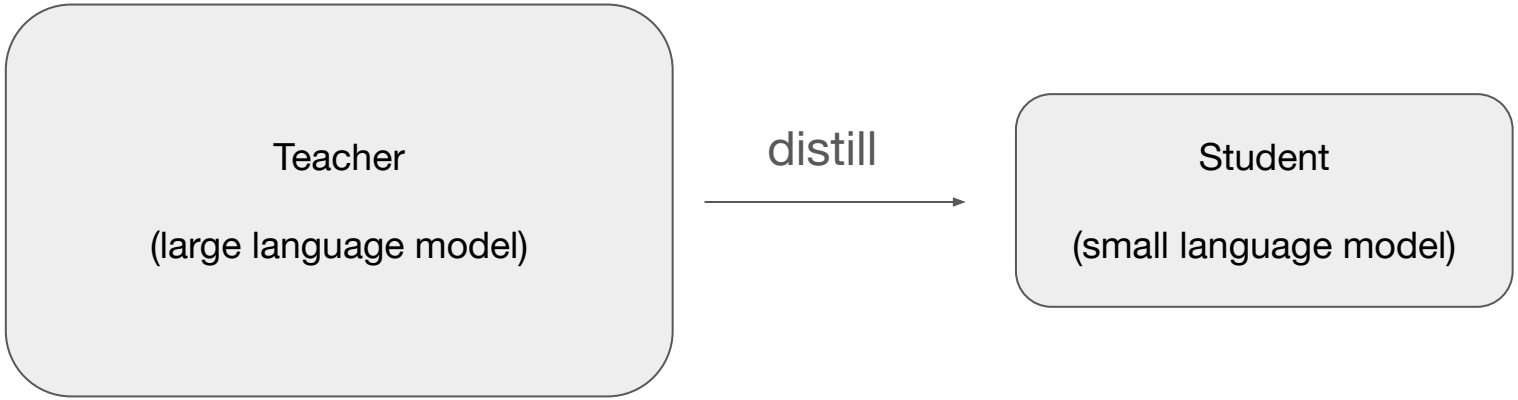
Fast Inference from Transformers via Speculative Decoding, Leviathan et al, 2023; https://arxiv.org/abs/2211.17192

Overview

- Training efficiency:
  - Parameter-efficient finetuning
    - Prefix tuning
    - LoRA
    - Adapters
- Inference efficiency:
  - Speculative decoding
  - Knowledge distillation

Knowledge distillation

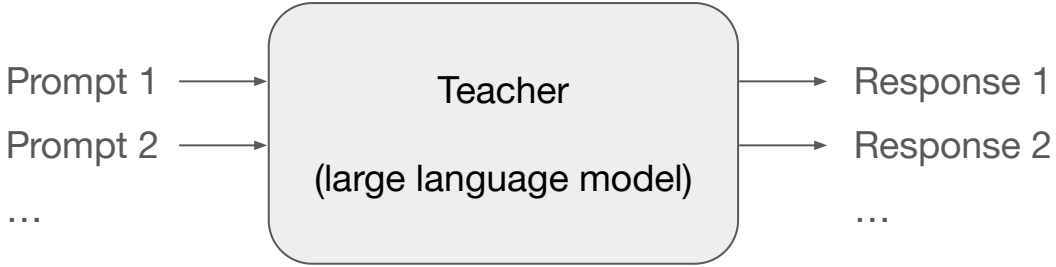
- Use a large model to generate data for training a smaller model



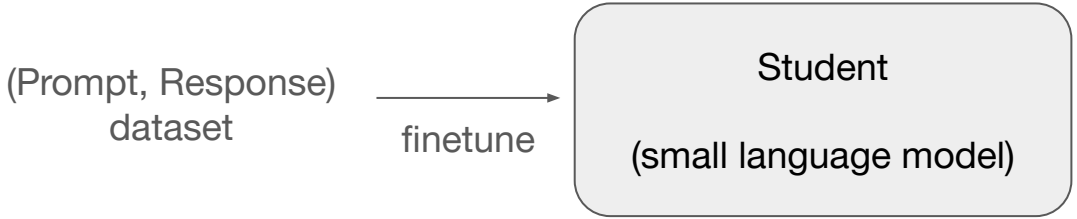
Distilling the Knowledge in a Neural Network, Hinton et al, 2015

Sequence-level distillation

- Get a large collection of prompts {x_i} and generate responses using the teacher model

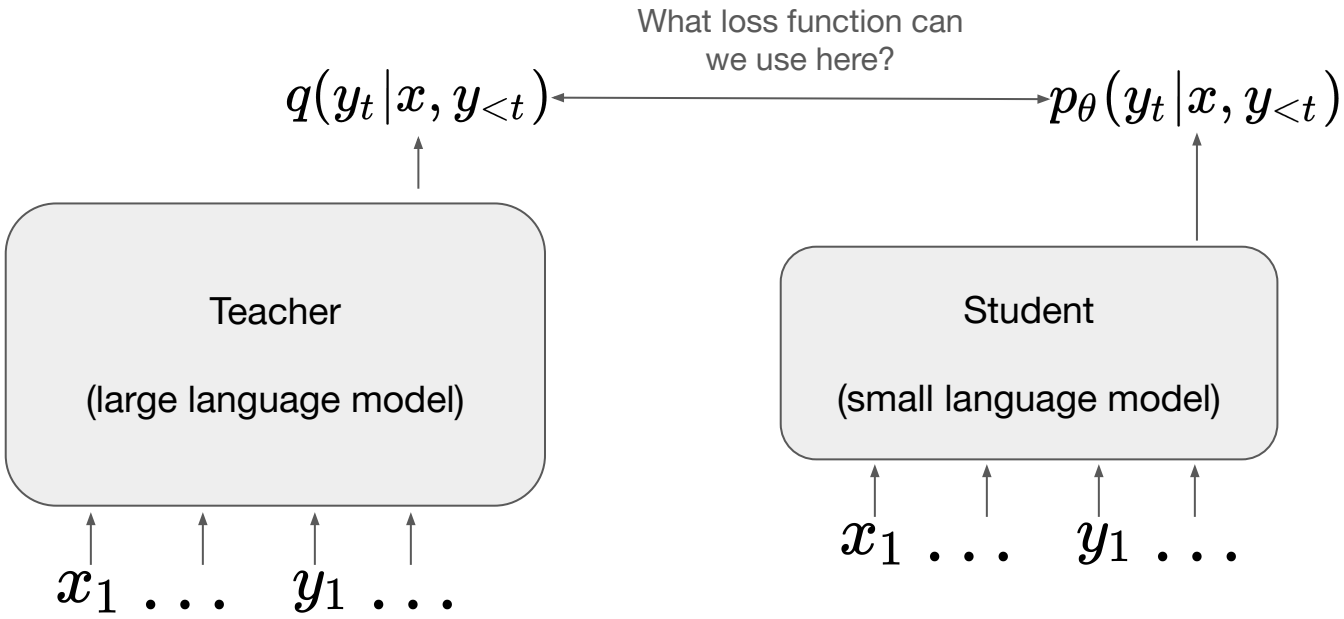


- Finetune the student model on the generated responses



Token-level distillation

- Train the student to match the teacher's token distribution

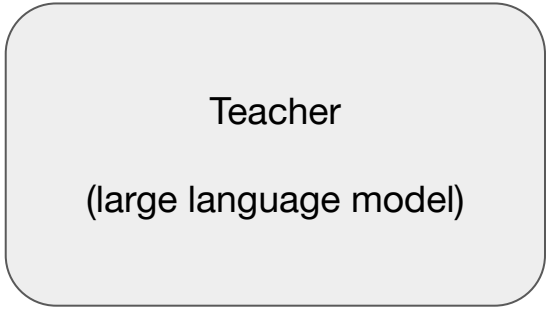


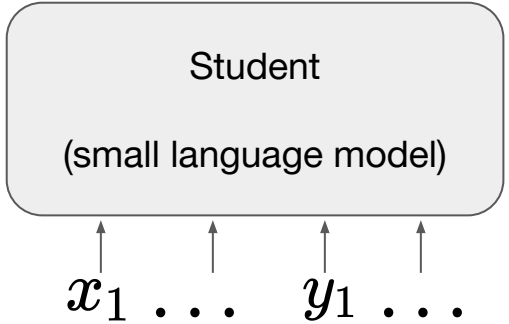
Token-level distillation

- Train the student to match the teacher's token distribution

$$-\sum_{y_t \in \mathcal{V}} q(y_t | x, y_{<t}) \log p_\theta(y_t | x, y_{<t})$$

cross-entropy!





What are we optimizing here?

$$\min_\theta KL(q(y|x)||p_\theta(y|x))$$

$$= \mathbb{E}_{y\sim q(\cdot|x)}\left[-\log p_\theta(y|x)\right] - H(q(y|x))$$

What are we optimizing here?

$$\min_\theta KL(q(y|x)||p_\theta(y|x))$$

$$= \mathbb{E}_{y\sim q(\cdot|x)}\left[-\log p_\theta(y|x)\right] - \cancel{H(q(y|x))}$$

does not depend on theta

Negative log-likelihood over teacher samples
(sequence-level distillation)

What are we optimizing here?

$$\min_\theta KL(q(y|x)||p_\theta(y|x))$$

$$= \sum_{t=1}^{T} \mathbb{E}_{y_{<t}\sim q}\left[KL(q(y_t|x, y_{<t})||p_\theta(y_t|x, y_{<t}))\right]$$

Sum of token-level KL divergences –
as long as *y* is sampled from *q*

What are we optimizing here?

$$\min_\theta KL(q(y|x)||p_\theta(y|x))$$

$$= \sum_{t=1}^{T} \mathbb{E}_{y_{<t}\sim q}\left[KL(q(y_t|x, y_{<t})||p_\theta(y_t|x, y_{<t}))\right]$$

$$= \mathbb{E}_{y_t\sim q(\cdot|x,y_{<t})}\left[-\log p_\theta(y_t|x, y_{<t})\right] - \cancel{H(q(y_t|x, y_{<t}))}$$

Sum of token-level KL divergences – as long as *y* is sampled from *q*

Both sequence-level and token-level distillation optimize KL divergence between teacher and student distributions (if responses are sampled from the teacher)

Summary

- For training efficiency:
  - Finetune an LLM using parameter-efficient finetuning
- For inference efficiency:
  - Use speculative decoding to offload generation to small model
  - Or distill the larger model into a smaller one